Guía de trabajo Evaluación Sumativa 3: SGR

Índice: quién hace qué

Juliet se encarga de la seguridad, de Usuarios y Metas, de los paneles y de AWS; Rocío, de sacar Django Admin de las pantallas de trabajo, de Atenciones sociales y Catálogo, de las transacciones y de la auditoría. La revisión es el 3 de noviembre.

Juliet (UsuarioApp, IndicadoresApp)

1. Primero, antes del 23 de octubre: grupos y permisos de Django
2. Proteger sus vistas con @login_required y permisos
3. Recuperación de contraseña por correo
4. Mantenedor 1: Usuarios, con asignación de grupo
5. Mantenedor 2: Metas
6. Paneles por perfil: Administrador, Supervisor y Administrativo
7. Sacar Django Admin de Semáforo, Resumen y Administración
8. Al final: instancia nueva en AWS con MySQL dentro

Rocío (ActividadesApp, AuditoriaApp)

1. Primero: inventario de botones que van a Django Admin
2. Componentes compartidos: barra_crud y acciones_fila
3. Proteger sus vistas con @login_required y permisos
4. Mantenedor 3: Atenciones sociales
5. Mantenedor 4: Catálogo de ítems y tipos
6. Transacciones: registrar actividad y validar evidencia
7. Auditoría de solo lectura con filtros
8. Armado del documento técnico con capturas

Las dos: reglas de trabajo en equipo (commits, tests, evidencia de IA) y calendario con lista final.

1. Cómo leer esta guía

El 3 de noviembre el profesor va a revisar el SGR funcionando, y esta guía dice qué cambia, por qué y quién lo hace. Está escrita para las dos integrantes del equipo: Juliet (UsuarioApp e IndicadoresApp) y Rocío (ActividadesApp y AuditoriaApp).

Léanla en este orden:

1. **Sección 2:** qué ya funciona y qué falta.
2. **Sección 3:** los conceptos que se usan en el resto de la guía. Si una palabra no se entiende, está explicada ahí.
3. **Sección 4:** la tarea que va primero, porque las demás dependen de ella.
4. **Secciones 5 y 6:** las tareas de cada una, paso a paso.
5. **Secciones 7 y 8:** cómo trabajar sin pisarse y el calendario.

Qué pide la Evaluación Sumativa 3

La organización quiere que el sistema funcione como una aplicación real: **todo se hace desde pantallas propias, sin Django Admin**. Además pide inicio de sesión con perfiles, grupos y permisos, recuperación de contraseña por correo y paneles distintos según el perfil.

Criterio	Puntos	En una frase
Sistema de autenticación	15	Entrar, salir y que ninguna página se vea sin sesión
Gestión de perfiles y permisos	15	Grupos de Django con permisos distintos
Recuperación de contraseña	15	Recibir un correo con un enlace y cambiar la clave
Dashboard por perfil	10	Un panel distinto para cada perfil
Mantenedores CRUD	20	Cuatro pantallas para crear, buscar, listar, modificar y eliminar
Transacciones	10	Dos operaciones que guardan datos en varias tablas relacionadas
Base de datos y ORM	5	Modelos, relaciones y migraciones
AWS EC2	5	El sistema corriendo en la nube

Criterio	Puntos	En una frase
GitHub	3	Historial de commits de las dos
Evidencia de IA	2	Prompts, respuestas y cómo se usaron

Por ser dos integrantes, la pauta exige **4 mantenedores y 2 transacciones**, no 2 y 1.

Regla de oro de esta evaluación: si el profesor aprieta un botón y la dirección del navegador muestra `/admin/`, esa operación no cuenta.

2. Diagnóstico: qué ya funciona y qué falta

El proyecto se ve completo en pantalla, pero tres de los criterios más pesados (45 de 100 puntos) necesitan trabajo nuevo. La revisión se hizo sobre los 82 archivos de código de la carpeta del proyecto.

Lo que ya cumple

- **Inicio y cierre de sesión** con pantalla propia (`users/login.html`).
- **Dos transacciones**: registrar una actividad con evidencia, y validar o rechazar esa evidencia. Ambas guardan datos en varias tablas relacionadas.
- **Modelos, relaciones y migraciones** con Django ORM y MySQL (WampServer, puerto 3306).
- **Variables de entorno** en el archivo `.env` (clave secreta, base de datos, `DEBUG`).
- **Permisos por rol** en las pantallas: cada botón solo lo ve quien puede usarlo.

Lo que falta

Problema	Dónde se ve	Por qué importa
33 botones que abren Django Admin	Agregar, Buscar, Modificar y Eliminar en 9 pantallas	La pauta prohíbe Django Admin
Los roles no usan grupos de Django	Tablas propias `Rol` y `UsuarioRol`	La pauta pide "asignación de grupos" y "sistema de permisos y grupos"
No hay recuperación de contraseña	No existe la pantalla ni el envío de correo	Vale 15 puntos

Problema	Dónde se ve	Por qué importa
Un solo panel igual para todos	`indicadores/dashboard.html`	Se piden tres paneles distintos
Mantenedores incompletos	Usuarios y Metas: buscar, modificar y eliminar van a Django Admin	Se piden cuatro completos
Las vistas se protegen "a mano"	Cada función de `views.py`	Es fácil olvidar una (sección 3.4)

Dato importante para la demostración: los usuarios de prueba de `datos_iniciales.sql` tienen correos `@laserena.cl`, que no existen. Para mostrar la recuperación de contraseña, al menos un usuario necesita un correo real que puedan abrir frente al profesor.

3. Conceptos base

Cada pantalla de Django se arma con tres piezas: una **ruta**, una **vista** y una **plantilla**. Si entienden estas tres, entienden todo lo que hay que cambiar.

3.1 Ruta, vista y plantilla

- **Ruta (URL):** la dirección que se escribe en el navegador. Se define en `urls.py` y conecta esa dirección con una vista.
- **Vista:** una función de Python en `views.py`. Decide quién puede entrar, consulta la base de datos y prepara los datos.
- **Plantilla (template):** el archivo HTML en `templates/` que muestra esos datos con Bootstrap.

Ejemplo real del proyecto, la pantalla Verificación:

```
# ActividadesApp/urls.py  (la ruta)
path('verificacion/', vactividades.verificacion, name="verificacion"),
```

```
# ActividadesApp/views.py  (la vista)
def verificacion(request):
    ...
    return render(request, 'actividades/verificacion.html', data)
```

La vista termina con `render`, que toma la plantilla `actividades/verificacion.html`, le pone los datos y devuelve la página terminada al navegador.

El `name="verificacion"` permite escribir el enlace en las plantillas sin copiar la dirección completa: `{% url 'actividades:verificacion' %}`.

3.2 Django Admin y rutas propias

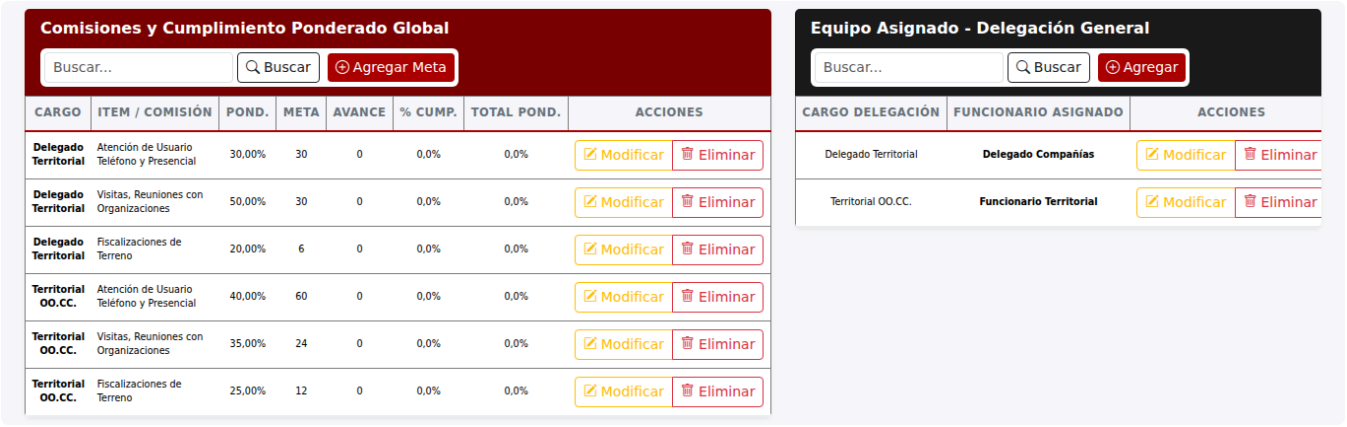
Django Admin es un panel que Django crea solo, en `/admin/`. Permite crear, editar y borrar registros de cualquier tabla sin programar pantallas. Es cómodo, pero genérico.

Hoy los botones del SGR apuntan a ese panel. Ejemplo real en `indicadores/resumen_delegacion.html`:

```
{% url 'admin:IndicadoresApp_metadesempeno_change' item.id as url_modificar
%}
{% include 'components/acciones_fila.html' with url_modificar=url_modificar
... %}
```

La palabra `admin:` al inicio indica que es una ruta de Django Admin. Al apretar Modificar, el navegador va a `/admin/IndicadoresApp/metadesempeno/5/change/`.

Así se ve hoy esa pantalla (Resumen). Todos los botones Buscar, Agregar, Modificar y Eliminar de la imagen abren Django Admin:



En la Evaluación 2 eso estaba permitido. **En esta evaluación está prohibido.**

Una **ruta propia** es una dirección que ustedes definen, con su vista y su plantilla. Por ejemplo `/indicadores/metas/5/editar/`, programada por el equipo. "Reemplazar por rutas propias" significa construir esa pantalla y hacer que el botón apunte a ella:

```
{% url 'indicadores:meta_editar' item.id as url_modificar %}
```

3.3 Mantenedor y transacción

- **Mantenedor:** pantalla para administrar los registros de una tabla con cinco operaciones: crear, buscar, listar, modificar y eliminar. También se le llama **CRUD** (*Create, Read, Update, Delete*). Ejemplo: el mantenedor de Usuarios.
- **Transacción:** una operación del trabajo diario que guarda datos en **dos o más tablas relacionadas** con validaciones. Ejemplo: registrar una actividad crea una fila en `actividad` y otra en `evidencia`, ligadas entre sí.

Lo que se debe modificar en Metas

Operación	Dónde se hace hoy	¿Cuenta para la pauta?
Crear	Formulario propio en Administración, pestaña Parametrización de Metas	Sí
Listar	Tabla "Metas del periodo vigente"	Sí
Modificar	Botón en Resumen y Mi Gestión que abre `/admin/`	No
Eliminar	Botón que abre `/admin/`	No
Buscar	Abre la lista de Django Admin	No

De las cinco operaciones, solo dos son pantallas propias. Para que Metas sea un mantenedor completo faltan las otras tres.

3.4 Decoradores: proteger las vistas

Hoy cada vista empieza revisando a mano si hay sesión y si el usuario tiene el rol correcto:

```
def verificacion(request):
    if not request.user.is_authenticated:
        return redirect('users:login')
    if not request.user.puede_verificar():
        return acceso_denegado(request, 'Verificación')
```

Funciona, pero si alguien crea una vista nueva y olvida copiar esas líneas, la vista queda abierta a cualquiera. Django resuelve esto con **decoradores**: una línea que empieza con `@` y se pone sobre la función.

```
from django.contrib.auth.decorators import login_required,
permission_required

@login_required
@permission_required('ActividadesApp.validar_evidencia',
raise_exception=True)
def verificacion(request):
    ...
```

- `@login_required`: si no hay sesión, manda al login. Usa `LOGIN_URL = 'users:login'`, que ya está en `config/settings.py`.
- `@permission_required(...)`: si el usuario no tiene ese permiso, responde "acceso denegado" (error 403).

3.5 Grupos y permisos

- **Permiso:** una autorización concreta, como "puede validar evidencias" o "puede modificar metas".
- **Grupo:** un perfil que reúne varios permisos. Al usuario se le asigna un grupo y hereda todos sus permisos.

Piénsenlo como una credencial: el grupo "Supervisor" trae una lista de puertas que puede abrir, y el usuario que recibe esa credencial abre esas puertas.

¿Cambia la base de datos? No se crean tablas nuevas. Django ya las creó en el primer `migrate` y hoy están vacías o casi vacías:

Tabla	Qué guarda
`auth_permission`	Todos los permisos que existen
`auth_group`	Los grupos
`auth_group_permissions`	Qué permisos tiene cada grupo
`usuario_groups`	Qué grupo tiene cada usuario

Lo que cambia son las **filas** dentro de esas tablas, no su estructura.

Dos tipos de permisos:

1. **Automáticos.** Django crea cuatro por cada modelo: `add_` (agregar), `change_` (modificar), `delete_` (eliminar) y `view_` (ver). Ejemplo: `IndicadoresApp.change_metadesempeno`. Ya existen, no hay que escribirlos.
2. **Propios**, para acciones que no son agregar, modificar, eliminar ni ver. Se declaran en el modelo:

```
# ActividadesApp/models.py
class Evidencia(models.Model):
    ...
    class Meta:
        permissions = [("validar_evidencia", "Puede validar o rechazar
evidencias")]
```

Esto genera una migración con `python manage.py makemigrations`, pero no cambia ninguna tabla: solo agrega una fila en `auth_permission`.

Dónde se usan los permisos:

Lugar	Cómo se escribe
Vista	`@permission_required('ActividadesApp.validar_evidencia', raise_exception=True)`
Plantilla	`{% if perms.ActividadesApp.validar_evidencia %} ... {% endif %}`
Código Python	`request.user.has_perm('ActividadesApp.validar_evidencia')`

El superusuario (Administrador) tiene todos los permisos automáticamente.

4. Primera tarea: grupos y permisos

Esta tarea va antes que todo porque las demás la usan: sin grupos no se pueden proteger las vistas ni armar los paneles por perfil. **La hace Juliet y debe quedar en `main` a más tardar el 23 de octubre.**

4.1 Los cinco perfiles

El SGR ya tiene cinco roles. Se mantienen, pero cada uno queda asociado a un grupo de Django con el nombre que usa la pauta:

Rol actual	Grupo de Django	Nivel de acceso
Admin	Administrador	Acceso total
Coordinador	Supervisor	Permisos intermedios
Funcionario	Administrativo	Permisos mínimos
Delegado	Delegado	Perfil extra: valida evidencias de su delegación
Consulta	Consulta	Perfil extra: solo mira

4.2 Tabla de permisos

Esta tabla es el contrato entre las dos. Revísenla juntas antes de escribir código: lo que se decida aquí es lo que cada vista va a exigir.

Permiso	Administrador	Supervisor	Delegado	Administrativo	Consulta
Ver actividades y metas (`view_*`)	Sí	Sí	Sí	Sí	Sí
Registrar actividad (`add_actividad` , `add_evidencia`)	Sí	Sí	No	Sí	No
Modificar o eliminar actividad	Sí	Sí	No	No	No
Validar evidencia (`validar_evidencia`)	Sí	Sí	Sí	No	No
Usuarios (`add_` , `change_` , `delete_usuario`)	Sí	Sí	No	No	No
Metas y Catálogo (agregar, modificar, eliminar)	Sí	Sí	No	No	No
Atenciones sociales (agregar)	Sí	Sí	No	Sí	No
Ver auditoría (`view_auditoria`)	Sí	Sí	No	No	No

Dos reglas siguen igual que hoy y se revisan dentro de la vista, no con permisos: el Supervisor no puede crear ni asignar Administradores o Supervisores, y solo el Administrador modifica fechas.

4.3 Paso a paso

Paso 1. El permiso propio de validar evidencia. En `ActividadesApp/models.py`, dentro de `class Evidencia`, agregar el `Meta` de la sección 3.5. Es una línea en un archivo de Rocío: avísenle antes. Luego:

```
python manage.py makemigrations ActividadesApp
python manage.py migrate
```

Paso 2. La equivalencia entre rol y grupo. En `UsuarioApp/choices.py`:

```
GRUPO_DE_ROL = {
    'Admin': 'Administrador',
    'Coordinador': 'Supervisor',
    'Delegado': 'Delegado',
    'Funcionario': 'Administrativo',
    'Consulta': 'Consulta',
}
```

Paso 3. Un comando que crea los grupos. Se crea el archivo `UsuarioApp/management/commands/crear_grupos.py`. Las carpetas `management` y `commands` necesitan cada una un archivo vacío `__init__.py`.

```
from django.contrib.auth.models import Group, Permission
from django.core.management.base import BaseCommand

from UsuarioApp.choices import GRUPO_DE_ROL
from UsuarioApp.models import UsuarioRol

# app.codigo de cada permiso, copiado de la tabla 4.2
PERMISOS = {
    'Supervisor': ['ActividadesApp.add_actividad',
'ActividadesApp.change_actividad',
                   'ActividadesApp.validar_evidencia',
'UsuarioApp.add_usuario',
                   'UsuarioApp.change_usuario',
'AuditoriaApp.view_auditoria'],  # ... completar
    'Delegado': ['ActividadesApp.view_actividad',
'ActividadesApp.validar_evidencia'],
    'Administrativo': ['ActividadesApp.add_actividad',
'ActividadesApp.add_evidencia'],
    'Consulta': ['ActividadesApp.view_actividad',
'IndicadoresApp.view_metadesempeno'],
```

```
}

class Command(BaseCommand):
    help = 'Crea los grupos del SGR, les asigna permisos y pone a cada
usuario en su grupo'

    def handle(self, *args, **options):
        admin, _ = Group.objects.get_or_create(name='Administrador')
        admin.permissions.set(Permission.objects.all())
        for nombre, lista in PERMISOS.items():
            grupo, _ = Group.objects.get_or_create(name=nombre)
            permisos = []
            for texto in lista:
                app, codigo = texto.split('.')

permisos.append(Permission.objects.get(content_type__app_label=app,
codename=codigo))
            grupo.permissions.set(permisos)
        # Los usuarios que ya existen quedan en el grupo de su rol
        for ur in UsuarioRol.objects.select_related('usuario', 'rol'):

ur.usuario.groups.set([Group.objects.get(name=GRUPO_DE_ROL[ur.rol.codigo])])
        self.stdout.write('Grupos listos')
```

Se ejecuta después de `migrate`, en el computador de cada una y después en AWS:

```
python manage.py crear_grupos
```

¿Por qué un comando y no una migración? En una base de datos nueva, Django crea los permisos *después* de correr todas las migraciones. Una migración que intente asignarlos puede no encontrarlos. El comando se corre cuando los permisos ya existen, y se puede repetir sin duplicar nada.

Paso 4. Asignar el grupo cuando se asigna el rol. En `UsuarioApp/views.py`, función `asignar_rol`, agregar al final:

```
from django.contrib.auth.models import Group
from UsuarioApp.choices import GRUPO_DE_ROL

    grupo = Group.objects.get(name=GRUPO_DE_ROL[codigo])
    usuario.groups.set([grupo])
```

¿Por qué no borrar `UsuarioRol`? Reemplazarlo obliga a reescribir decenas de condiciones y tests a pocos días de la entrega. Además guarda quién asignó el rol y cuándo, cosa que los grupos de Django no registran. Es una buena respuesta si el profesor pregunta por qué existen las dos cosas.

Paso 5. Que los métodos de permiso consulten los grupos. En `UsuarioApp/models.py`, clase `Usuario`:

```
# Antes: revisaba nombres de rol
def puede_verificar(self):
    return self.tiene_rol(ROLES_VERIFICACION)

# Después: revisa el permiso del grupo
def puede_verificar(self):
    return self.has_perm('ActividadesApp.validar_evidencia')
```

Lo mismo con `puede_administrar` (`UsuarioApp.change_usuario`) y `puede_registrar` (`ActividadesApp.add_actividad`). Como las plantillas ya usan `request.user.puede_verificar`, siguen funcionando sin tocarlas.

Paso 6. El formulario de usuario muestra "Perfil (grupo)". En `users/usuario_form.html`, el campo "Rol" pasa a llamarse "Perfil (grupo)". Así, en la revisión, crear un usuario demuestra al mismo tiempo "creación de usuarios" y "asignación de grupos".

Paso 7. Arreglar los tests. Los tests crean usuarios con rol, pero ahora los permisos vienen de los grupos. En cada clase de test, agregar al inicio de `setUp`:

```
from django.core.management import call_command

def setUp(self):
    call_command('crear_grupos')
    ...
```

4.4 Cómo saber que quedó bien

- [ ] `python manage.py crear_grupos` termina diciendo "Grupos listos".
- [ ] En phpMyAdmin, la tabla `auth_group` tiene 5 filas y `usuario_groups` tiene una fila por usuario.
- [ ] Al crear un usuario con perfil Supervisor, aparece en `usuario_groups` con el grupo Supervisor.
- [ ] `python manage.py test` pasa completo.

5. Tareas de Juliet: UsuarioApp, IndicadoresApp y AWS

Juliet se encarga de la seguridad (grupos, recuperación de contraseña), de dos mantenedores, de los paneles por perfil y, al final, de AWS. Lo primero es la sección 4; después, estas tareas en orden.

5.1 Proteger las vistas de sus apps

Qué cambia: cada vista de `UsuarioApp/views.py` e `IndicadoresApp/views.py` pasa a usar decoradores en vez de revisar a mano (sección 3.4).

```
# Antes
def administracion(request):
    if not request.user.is_authenticated:
        return redirect('users:login')
    if not request.user.puede_administrar():
        return acceso_denegado(request, 'Administración')
    ...

# Después
@login_required
@permission_required('UsuarioApp.change_usuario', raise_exception=True)
def administracion(request):
    ...
```

Vistas a revisar: `administracion`, `usuario_crear`, `usuario_editar`, `dashboard`, `semaforo`, `resumen_delegacion` y `guardar_metas`. El login y el logout quedan sin `@login_required`, porque se usan sin sesión.

5.2 Recuperación de contraseña por correo

Qué es: si alguien olvida su clave, escribe su correo, recibe un enlace único y desde ese enlace elige una clave nueva. Django trae las cuatro vistas hechas; hay que conectarlas, darles plantillas propias y configurar el correo.

El flujo que se demuestra en la revisión:

1. En el login, el usuario aprieta "¿Olvidaste tu contraseña?" y escribe su correo.
2. Django genera un **token**: un código secreto, de un solo uso y con vencimiento, que solo sirve para ese usuario.
3. Django envía al correo un enlace que contiene ese token.
4. Al abrir el enlace, Django **valida el token**. Si es falso o venció, no deja seguir.

5. El usuario escribe la clave nueva dos veces.
6. Django confirma el cambio y el usuario entra con la clave nueva.

Paso 1. Configurar el correo en `config/settings.py`. Las credenciales van en `.env`, nunca en el código:

```
EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST = 'smtp.gmail.com'
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_HOST_USER = os.getenv('EMAIL_HOST_USER')
EMAIL_HOST_PASSWORD = os.getenv('EMAIL_HOST_PASSWORD')
DEFAULT_FROM_EMAIL = EMAIL_HOST_USER
PASSWORD_RESET_TIMEOUT = 3600  # el enlace vence en 1 hora
```

En Gmail, `EMAIL_HOST_PASSWORD` no es la clave normal de la cuenta: es una **contraseña de aplicación**, que se genera en la configuración de seguridad de Google (requiere verificación en dos pasos).

Mientras se programa, se puede usar `EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'`: el correo se imprime en la terminal en vez de enviarse.

Paso 2. Las rutas, en `UsuarioApp/urls.py`:

```
from django.contrib.auth import views as auth_views
from django.urls import reverse_lazy

path('recuperar/', auth_views.PasswordResetView.as_view(
        template_name='users/recuperar.html',
        email_template_name='users/recuperar_email.txt',
        success_url=reverse_lazy('users:recuperar_enviado')),
name='recuperar'),
path('recuperar/enviado/', auth_views.PasswordResetDoneView.as_view(
        template_name='users/recuperar_enviado.html'),
name='recuperar_enviado'),
path('recuperar/<uidb64>/<token>/',
auth_views.PasswordResetConfirmView.as_view(
        template_name='users/recuperar_nueva.html',
        success_url=reverse_lazy('users:recuperar_listo')),
name='password_reset_confirm'),
path('recuperar/listo/', auth_views.PasswordResetCompleteView.as_view(
        template_name='users/recuperar_listo.html'), name='recuperar_listo'),
```

¿Por qué `success_url`? Las rutas del proyecto tienen apellido (`users:`). Las vistas de Django buscan por defecto rutas sin apellido y fallarían con un error `NoReverseMatch`. Con `success_url` se les dice exactamente a dónde ir.

Paso 3. Las plantillas. Cinco archivos en `templates/users/`, con el mismo estilo del login:

Archivo	Qué muestra
`recuperar.html`	Campo para escribir el correo
`recuperar_email.txt`	El texto del correo con el enlace
`recuperar_enviado.html`	"Si el correo existe, te enviamos un enlace"
`recuperar_nueva.html`	Los dos campos de la clave nueva
`recuperar_listo.html`	"Clave cambiada" y botón para ir al login

El correo arma el enlace así:

```
{{ protocol }}://{{ domain }}{% url 'users:password_reset_confirm' uidb64=uid
token=token %}
```

Paso 4. El enlace en el login. En `users/login.html`, bajo el botón Ingresar:

```
<a href="{% url 'users:recuperar' %}">¿Olvidaste tu contraseña?</a>
```

Detalle de seguridad para explicar en la revisión: la pantalla dice "si el correo existe, te enviamos un enlace" aunque el correo no esté registrado. Así nadie puede usar el formulario para averiguar qué correos tienen cuenta.

5.3 Mantenedor 1: Usuarios

Hoy crear y editar ya son pantallas propias. Faltan **buscar** y **eliminar**, que van a Django Admin.

Buscar. En la vista `administracion`, filtrar la lista por lo que se escriba en el buscador:

```
from django.db.models import Q

q = request.GET.get('q', '').strip()
usuarios = Usuario.objects.all()
if q:
    usuarios = usuarios.filter(Q(username__icontains=q) |
Q(first_name__icontains=q) |
                               Q(last_name__icontains=q) |
Q(email__icontains=q))
```

`Q` permite buscar en varias columnas a la vez; `icontains` busca el texto en cualquier parte, sin importar mayúsculas.

Eliminar (desactivar). Una vista `usuario_desactivar` con una pantalla de confirmación que marca `is_active = False`. Se recomienda desactivar en vez de borrar: el usuario tiene actividades, validaciones y auditoría asociadas, y borrarlo dejaría esos registros sin dueño. Un usuario inactivo no puede iniciar sesión.

Regla: borrar o desactivar se hace siempre con un formulario `POST` y un botón de confirmación, nunca con un enlace directo. Un enlace se puede abrir por error; un formulario exige una decisión.

5.4 Mantenedor 2: Metas

Faltan **buscar, modificar y eliminar** (sección 3.3).

Paso 1. Un formulario con las validaciones dentro. Hoy las reglas del ponderador (mayor que 0, máximo 100, la suma del cargo no supera 100%) están escritas dentro de la vista `guardar_metas`. Si se pasan a un formulario de Django, valen igual para crear y para

modificar:

```
# IndicadoresApp/forms.py  (archivo nuevo)
class MetaForm(forms.ModelForm):
    class Meta:
        model = MetaDesempeno
        fields = ['cargo', 'item', 'valor_objetivo', 'ponderador']

    def clean(self):
        datos = super().clean()
        # aquí van las mismas reglas que hoy están en guardar_metas
        return datos
```

Paso 2. Cuatro vistas y sus rutas: `meta_lista` (con buscador por cargo o ítem), `meta_crear`, `meta_editar` y `meta_eliminar` (con confirmación). La vista `meta_editar` está completa en la sección 3.2.

Paso 3. Cambiar los botones de `indicadores/resumen_delegacion.html` para que apunten a `indicadores:meta_editar` y `indicadores:meta_eliminar` en vez de `admin:...`.

La tabla de metas de `actividades/mi_gestion.html` (líneas 104 a 135) usa los mismos botones, pero ese archivo es de Rocío: cuando las rutas existan, Juliet le pasa los nombres y Rocío los cambia.

5.5 Dashboards por perfil

Hoy existe un solo panel (`indicadores/dashboard.html`) igual para todos. La pauta pide tres distintos.

Cómo funciona: la vista `dashboard` mira el grupo del usuario y elige qué plantilla mostrar y qué datos preparar.

```
@login_required
def dashboard(request):
    usuario = request.user
    if usuario.is_superuser or
usuario.groups.filter(name='Administrador').exists():
        plantilla = 'indicadores/dashboard_admin.html'
    elif usuario.groups.filter(name__in=['Supervisor', 'Delegado']).exists():
        plantilla = 'indicadores/dashboard_supervisor.html'
    else:
        plantilla = 'indicadores/dashboard_administrativo.html'
    ...
    return render(request, plantilla, data)
```

Qué muestra cada uno, según la pauta:

Panel	Contenido
Administrador	Accesos a los 4 mantenedores y a las 2 transacciones, gestión de usuarios, totales del sistema (usuarios, actividades, evidencias pendientes, semáforo global)
Supervisor y Delegado	Accesos a lo autorizado (Verificación, Metas, Semáforo) e información operacional de su delegación
Administrativo y Consulta	Solo lo propio: sus actividades, sus compromisos del Tubo, su avance. Sin botones de modificar ni eliminar

Cada acceso del panel es una tarjeta de Bootstrap con un enlace, visible solo si el usuario tiene el permiso (`{% if perms... %}`).

5.6 Sacar Django Admin de sus pantallas

Pantalla	Botón	Qué hacer
`indicadores/resumen_delegacion.html`	Agregar, Buscar, Modificar y Eliminar meta	Apuntar al mantenedor de Metas

Pantalla	Botón	Qué hacer
`indicadores/resumen_delegacion.html`	Equipo asignado: agregar, modificar y eliminar usuario	Apuntar al mantenedor de Usuarios
`users/administracion.html`	Modificar y eliminar usuario	Apuntar a `usuario_editar` y `usuario_desactivar`
`indicadores/semaforo.html`	Buscar, Modificar y Eliminar indicador	**Quitar**: los indicadores se calculan solos, no se editan
`indicadores/semaforo.html`	Agregar ajuste	Quitar, o hacer un mantenedor de Ajustes si sobra tiempo

Quitar un botón que no tiene sentido no resta puntos. Dejar un botón que lleva a `/admin/` sí resta.

5.7 AWS: al final, cuando el proyecto esté listo

Se crea una instancia EC2 nueva con la base de datos **dentro de la misma instancia** (MySQL instalado en ella, el mismo motor que usan en WampServer, así el proyecto se comporta igual en el computador y en la nube). En ese caso `DB_HOST=127.0.0.1` es correcto: ese "localhost" es el de la instancia en la nube, no el del computador.

1. Crear la instancia. Ubuntu Server 24.04, tipo `t2.micro` o `t3.micro`. Grupo de seguridad:

- puerto 22 (SSH) solo desde "Mi IP";
- puerto 80 (web) desde cualquier lugar;
- **no abrir el 3306 (base de datos) ni el 8000**: la aplicación y la base conversan dentro de la misma máquina.

La IP elástica se mueve desde la consola de EC2: Elastic IPs, *Disassociate* de la instancia antigua y *Associate* a la nueva. La antigua se detiene, pero no se termina hasta que la nueva funcione.

2. Instalar lo necesario (conectado por SSH):

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3-venv python3-dev build-essential pkg-config
default-libmysqlclient-dev mysql-server nginx git
sudo mysql_secure_installation
```

3. Crear la base de datos y su usuario. Se entra con `sudo mysql` y se ejecuta:

```
CREATE DATABASE municipalidad CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
CREATE USER 'sgr_app'@'localhost' IDENTIFIED BY 'UNA_CLAVE_LARGA';
GRANT ALL PRIVILEGES ON municipalidad.* TO 'sgr_app'@'localhost';
GRANT ALL PRIVILEGES ON test_municipalidad.* TO 'sgr_app'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

`sgr_app` es un usuario solo para la aplicación: Django nunca debe conectarse como `root`. El permiso sobre `test_municipalidad` sirve para correr los tests en el servidor.

4. Bajar el proyecto, prepararlo y cargar datos:

```
cd ~ && git clone https://github.com/rocio-bvl/Back-End-2026.git && cd Back-
End-2026
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt gunicorn
cp .env.example .env && nano .env
python manage.py migrate
python manage.py crear_grupos
mysql -u sgr_app -p municipalidad < datos_iniciales.sql
python manage.py crear_grupos
python manage.py collectstatic --noinput
```

`crear_grupos` se corre dos veces: antes de cargar los datos crea los grupos, y después pone a los usuarios de prueba en su grupo.

En el `.env` del servidor: `DEBUG=False`, `ALLOWED_HOSTS` con la IP elástica, `DB_HOST=127.0.0.1`, `DB_USER=sgr_app`, una `SECRET_KEY` nueva y las credenciales del correo.

5. Gunicorn, el programa que mantiene Django corriendo aunque se cierre la conexión SSH. Archivo `/etc/systemd/system/sgr.service`:

```
[Unit]
Description=SGR Django
After=network.target mysql.service

[Service]
User=ubuntu
Group=www-data
WorkingDirectory=/home/ubuntu/Back-End-2026
ExecStart=/home/ubuntu/Back-End-2026/venv/bin/gunicorn --workers 3 --bind
127.0.0.1:8000 config.wsgi:application
Restart=always

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload && sudo systemctl enable --now sgr
```

6. Nginx, el programa que recibe las visitas en el puerto 80. Con `DEBUG=False`, Django deja de entregar los CSS y las fotografías, y Nginx se encarga de eso. Archivo `/etc/nginx/sites-available/sgr`:

```
server {
    listen 80;
    server_name TU_IP_ELASTICA;
    client_max_body_size 6M;
    location /static/ { alias /home/ubuntu/Back-End-2026/staticfiles/; }
    location /media/  { alias /home/ubuntu/Back-End-2026/media/; }
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/sgr /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default
sudo chmod 755 /home/ubuntu
sudo nginx -t && sudo systemctl restart nginx
```

`client_max_body_size 6M` es obligatorio: Nginx rechaza por defecto archivos de más de 1 MB, y las evidencias pueden pesar hasta 5 MB.

7. Probar el correo desde AWS. AWS limita el puerto 25, por eso se usa Gmail por el 587. La prueba final es la recuperación de contraseña completa, entrando por la IP elástica.

Costo: AWS cobra por las IP públicas, incluida la elástica, aunque la instancia esté detenida. Conviene revisarlo en la consola antes de dejar dos instancias.

6. Tareas de Rocío: ActividadesApp y AuditoriaApp

Rocío se encarga de sacar Django Admin de las pantallas de trabajo diario, de dos mantenedores, de las dos transacciones y de la auditoría. Puede empezar el mismo día con 6.1 y 6.2, que no dependen de los grupos; desde el 24 de octubre sigue con el resto.

6.1 Inventario de botones (empieza el primer día)

Antes de cambiar código, conviene decidir qué pasa con cada botón que hoy abre Django Admin en sus pantallas. Para cada uno hay dos caminos: **construir la ruta propia** o **quitar el botón** si no tiene sentido.

Pantalla	Botón que hoy va a `/admin/`	Decisión	Por qué
`actividades/mi_gestion.html`	Buscar actividades	Buscador propio en la misma página	Es parte de la transacción
`actividades/mi_gestion.html`	Modificar y Eliminar actividad	Rutas propias `actividad_editar` y `actividad_eliminar`	Solo Administrador y Supervisor
`actividades/mi_gestion.html`	Agregar, Modificar y Eliminar meta	Cambiar a las rutas de Juliet	Las metas son de su app (5.4)
`actividades/tubo_trabajo.html`	Buscar, Modificar y Eliminar compromiso	Igual que las actividades	Un compromiso es una actividad
`actividades/gestion_social.html`	Agregar, Buscar, Modificar y Eliminar atención	Mantenedor de Atenciones (6.4)	Hoy no hay ninguna pantalla propia
`actividades/verificacion.html`	Agregar evidencia	**Quitar**	La evidencia se sube desde la actividad

Pantalla	Botón que hoy va a `/admin/`	Decisión	Por qué
`actividades/verificacion.html`	Modificar y Eliminar evidencia	**Quitar**	Una evidencia rechazada se reemplaza resubiendo; se conserva el historial
`auditoria/components/tabla_logs.html`	Agregar, Modificar y Eliminar registro	**Quitar**	Una bitácora de auditoría no se edita (6.7)
`auditoria/components/tabla_logs.html`	Buscar	Filtros propios en la misma página	Ver 6.7

6.2 Los componentes compartidos

Casi todas las pantallas usan dos piezas reutilizables, que son de Rocío:

- `components/barra_crud.html`: el buscador y el botón Agregar sobre una tabla.
- `components/acciones_fila.html`: los botones Modificar y Eliminar de cada fila.

Estas piezas no tienen enlaces fijos: reciben las direcciones desde cada pantalla (`url_buscar`, `url_agregar`, `url_modificar`, `url_eliminar`). Por eso el cambio principal se hace en cada pantalla, en la línea que arma la dirección:

```
{# Antes: Django Admin #}
{% url 'admin:ActividadesApp_actividad_change' actividad.id as url_modificar
%}

{# Después: ruta propia #}
{% url 'actividades:actividad_editar' actividad.id as url_modificar %}
```

En los componentes cambian dos cosas:

1. **El buscador** hoy solo aparece para Administrador y Supervisor, porque llevaba a Django Admin. Ahora busca dentro de la misma página, así que puede verlo cualquiera que vea la tabla. El formulario queda con `action=""` y envía `?q=texto`; la vista filtra con ese texto, igual que en 5.3.
2. **Las condiciones** pasan de `request.user.puede_administrar` a permisos concretos cuando se necesite precisión, por ejemplo `perms.ActividadesApp.delete_actividad`.

6.3 Proteger sus vistas

Igual que en 5.1, pero en `ActividadesApp/views.py` y `AuditoriaApp/views.py`. Cada vista recibe `@login_required` y el permiso que corresponda según la tabla 4.2.

Vista	Permiso
`actividad_crear`, `subir_evidencia`	`ActividadesApp.add_actividad`
`verificacion`, `validar_evidencia`	`ActividadesApp.validar_evidencia`
`mi_gestion`, `tubo_trabajo`, `actividad_detalle`	`ActividadesApp.view_actividad`
`logs` (auditoría)	`AuditoriaApp.view_auditoria`

Lo que se queda dentro de la vista: las reglas que dependen del registro concreto, no del perfil. Por ejemplo: el Delegado solo valida evidencias de su delegación, nadie valida su propia evidencia y solo el dueño sube la evidencia de su actividad. Un permiso dice "puede validar"; la vista decide "puede validar *esta*".

6.4 Mantenedor 3: Atenciones sociales

Hoy toda la gestión social se hace en Django Admin. Hay que construir las cinco operaciones.

Cómo están hechos los datos: cada `AtencionSocial` pertenece a una `Actividad` (uno a uno) y puede tener hasta 3 `GestionSocial` (la 1.ª, 2.ª y 3.ª gestión de la tabla).

Paso 1. El formulario, en `ActividadesApp/forms.py` (archivo nuevo):

```
class AtencionSocialForm(forms.ModelForm):
    class Meta:
        model = AtencionSocial
        fields = ['actividad', 'nombre_solicitante', 'rut_solicitante',
'telefono_solicitante',
                  'requiere_visita', 'tipo_atencion', 'subtipo_atencion']
```

Para las tres gestiones se usa un **formset en línea**: un grupo de formularios hijos que se guardan junto al padre.

```
from django.forms import inlineformset_factory

GestionFormSet = inlineformset_factory(AtencionSocial, GestionSocial,
                                       fields=['numero_gestion', 'tipo',
'accion', 'fecha'],
                                       extra=3, max_num=3)
```

Paso 2. Cuatro vistas y sus rutas: `atencion_lista` (la tabla actual de `gestion_social.html`, más el buscador por nombre o RUT del solicitante), `atencion_crear`, `atencion_editar` y `atencion_eliminar` (con confirmación).

Paso 3. Validaciones a incluir: el RUT del solicitante debe ser válido (ya existe `validar_rut` en `UsuarioApp/funciones.py`) y la actividad elegida no puede tener ya otra atención.

6.5 Mantenedor 4: Catálogo de ítems y tipos

El catálogo guarda las listas de opciones del sistema: ítems de gestión, tipos y subtipos de atención, tipos de ajuste. Hoy solo se editan en Django Admin.

Excepción a la división por apps: el modelo `Catalogo` está en UsuarioApp (de Juliet), pero lo usan casi solo Actividades y Gestión Social. Rocío crea sus vistas en `ActividadesApp` y no toca `UsuarioApp/models.py`.

Las cinco operaciones:

- **Listar y buscar:** tabla con filtro por tipo y buscador por nombre.
- **Crear y modificar:** un `ModelForm` con `tipo`, `nombre`, `valor` y `estado`.
- **Eliminar:** usar el método `desactivar()` que ya tiene el modelo. Borrar de verdad fallaría, porque las actividades y atenciones que usan esa opción la protegen (`on_delete=models.PROTECT`), y Django muestra un error.

6.6 Las dos transacciones

Las dos ya funcionan desde pantallas propias. Hay que terminarlas.

Transacción 1: registrar actividad con evidencia (tablas `actividad`, `evidencia`, `catalogo`, `usuario`).

Hoy la vista `actividad_crear` lee el formulario campo por campo con `request.POST.get(...)`, y la plantilla tiene las opciones de ítem escritas a mano:

```
<option value="1">Atención de Usuario Teléfono y Presencial</option>
<option value="2">Visitas, Reuniones con Organizaciones</option>
<option value="3">Fiscalizaciones de Terreno</option>
```

La pauta pide **formularios Django**. Conviene crear un `ActividadForm` (`ModelForm`): las opciones salen de la base de datos y las validaciones quedan en el formulario. Si en AWS los ítems tuvieran otros números de id, la lista escrita a mano guardaría el ítem equivocado; el formulario lo evita.

Además faltan `actividad_editar` y `actividad_eliminar`, solo para Administrador y Supervisor (6.1).

Transacción 2: validar o rechazar evidencia (tablas `evidencia`, `validacion`, `actividad`). Ya está completa. Solo hay que protegerla con el permiso `validar_evidencia` y quitar los botones de 6.1.

En la revisión, cada transacción se demuestra de punta a punta: llenar el formulario, guardar y mostrar en phpMyAdmin las filas nuevas en cada tabla relacionada.

6.7 Auditoría de solo lectura

Hoy la tabla de auditoría muestra Agregar, Modificar y Eliminar (solo para el Administrador). Una bitácora de auditoría existe para demostrar quién hizo qué; si se puede editar, deja de servir como prueba. Es una pregunta de seguridad que el profesor puede hacer.

- Quitar los tres botones de `auditoria/components/tabla_logs.html`.
- Agregar filtros en la misma página: por usuario, por tipo de evento y por rango de fechas, con un formulario `GET` y `.filter(...)` en la vista `logs`.
- Proteger la vista con `AuditoriaApp.view_auditoria`.

7. Reglas de trabajo en equipo

La mayoría de los problemas en equipo no vienen del código, sino de dos personas cambiando el mismo archivo al mismo tiempo. Estas reglas lo evitan.

7.1 Dueñas de cada archivo

Archivo o carpeta	Dueña
`UsuarioApp/` (salvo lo indicado abajo), `IndicadoresApp/`, `config/settings.py`	Juliet
`ActividadesApp/`, `AuditoriaApp/`, `templates/base.html`, `templates/components/`	Rocío
`templates/users/`, `templates/indicadores/`	Juliet
`templates/actividades/`, `templates/auditoria/`	Rocío

Si una necesita cambiar un archivo de la otra, se lo pide y espera. Hay dos casos ya conocidos:

- Juliet agrega el permiso `validar_evidencia` en `ActividadesApp/models.py` (4.3, paso 1), avisándole antes.
- Rocío cambia los botones de metas en `mi_gestion.html` cuando Juliet le pase los nombres de sus rutas (5.4).

7.2 Acuerdos antes de escribir código

- [ ] Revisar juntas la tabla de permisos (4.2).
- [ ] Nombres de rutas con el patrón `app:entidad_accion`. Ejemplos: `users:usuario_lista`, `indicadores:meta_editar`, `actividades:atencion_crear`.
- [ ] Nombres de plantillas con el mismo patrón: `entidad_lista.html`, `entidad_form.html` (sirve para crear y modificar), `entidad_confirmar_eliminar.html`.

7.3 Ramas, commits y Pull Requests

Cada una trabaja en su rama (`juliet` y rocío), como hasta ahora.

1. **Antes de empezar cada día**, traer lo nuevo de `main` a su rama:

```
git checkout <su-rama>
git pull origin main
```

2. **Un commit por cada cosa que funciona**, con un mensaje que diga qué hace: "Mantenedor de Metas: editar y eliminar". La pauta revisa el historial de commits de las dos.
3. **Un Pull Request a `main` cada vez que una tarea esté terminada**, no al final. La otra lo revisa antes de unirlo.
4. **Nunca** usar `git push --force`, `git reset --hard` ni `git checkout -- .`. Pueden borrar trabajo sin forma de recuperarlo.

7.4 Tests

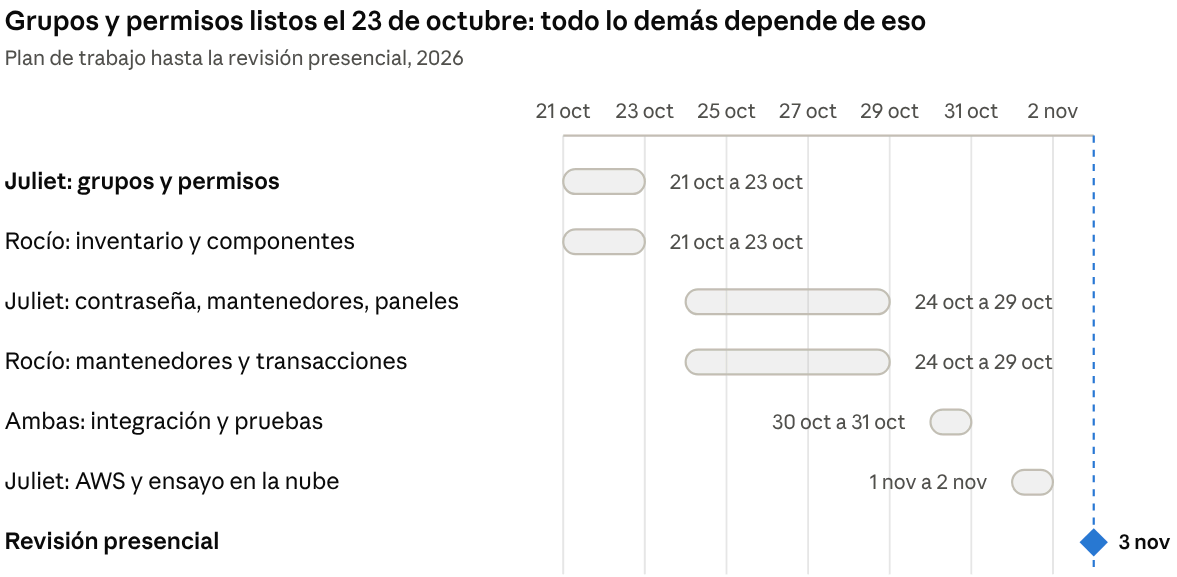
Antes de cada Pull Request, correr `python manage.py test`. Si algo falla, se arregla antes de unir. Cada una arregla los tests de su app; los de grupos y permisos los ajusta Juliet (4.3, paso 7).

7.5 Evidencia del uso de IA

La pauta pide los prompts usados, las respuestas obtenidas y cómo se aplicaron. Cada una guarda, a medida que avanza, capturas o copias de sus conversaciones con IA y anota en qué parte del código se usó cada respuesta. Esta guía y la conversación que la originó sirven como evidencia.

8. Calendario y lista final

Quedan 13 días, y el orden importa más que la velocidad: si los grupos y permisos no están en `main` el 23 de octubre, Rocío no puede proteger sus vistas y todo se atrasa en cadena.



plan de trabajo · 21 de octubre al 3 de noviembre

Los tres primeros días cada una trabaja en algo que no depende de la otra; desde el 24 se desarrolla en paralelo, y AWS queda para el final, cuando el proyecto ya funciona en local.

Lista para la revisión presencial

Revisen esta lista el 2 de noviembre, entrando por la IP elástica y no por el computador local.

Seguridad y acceso

- [ ] Inicio y cierre de sesión funcionan.
- [ ] Una página interna abierta sin sesión manda al login.
- [ ] Crear un usuario y asignarle un grupo desde pantallas propias.
- [ ] Entrar con cada perfil y mostrar que ve cosas distintas.
- [ ] Un Administrativo que escribe a mano la dirección de Verificación recibe "acceso denegado".

Recuperación de contraseña

- [ ] Pedir la recuperación con un correo real.
- [ ] Abrir el correo recibido y el enlace.
- [ ] Cambiar la clave y entrar con la clave nueva.

Paneles, mantenedores y transacciones

- [ ] Tres paneles distintos: Administrador, Supervisor y Administrativo.
- [ ] Cuatro mantenedores con crear, buscar, listar, modificar y eliminar: Usuarios, Metas, Atenciones sociales y Catálogo.
- [ ] Dos transacciones de punta a punta, mostrando las filas nuevas en la base de datos.
- [ ] Ningún botón del sistema lleva a `/admin/`.

Base de datos, GitHub y AWS

- [ ] Modelos, relaciones y migraciones explicados.
- [ ] Repositorio actualizado, con commits de las dos y `.gitignore` sin `.env`.
- [ ] Aplicación corriendo en EC2 y accesible desde el navegador.
- [ ] Evidencia de IA reunida para el documento técnico.